



**YILDIZ TECHNICAL UNIVERSITY
FACULTY OF CHEMICAL METALLURGICAL
DEPARTMENT OF MATHEMATICAL ENGINEERING**

**TECHNICAL REPORT OF
STRAY ANIMAL FOSTER & HEALTH TRACKING SYSTEM**

Instructor: Lecturer Dr. Fettah KIRAN

Authors:

Deniz YILMAZ 20058023

Batuhan TEKİNALP 21058037

Arif Emre POLAT 22052058

Damla Hilal ERDEN 22052088

PROJECT RESOURCES

Live Application:

<https://mtm4692-stray-animal-system-8efgqjfak7errdpk7ozwza.streamlit.app/>

GitHub Repository:

<https://github.com/denizyllm/MTM4692-Stray-Animal-System>

Demo Video:

https://drive.google.com/drive/folders/1gItPUUw14_j682JKSe-UMilgwsm74RG

1. INTRODUCTION

1.1 Problem Statement

Stray animal management by local municipalities and NGOs often suffers from fragmented data systems. This disconnection leads to critical operational failures:

- **Overcrowded Fosters:** Volunteers receiving more animals than their safe capacity.
- **Missed Medical Dates:** Lost track of crucial vaccination and treatment schedules.
- **Unassigned Donations:** Inefficient tracking of supplies (food, medicine) donated to specific volunteers.
- **Adoption Bottlenecks:** Lack of centralized tracking for adoption requests and animal statuses.

1.2. The Solution:

This project designs a normalized, relational database system that acts as a central coordination layer. It seamlessly connects animals, volunteers, clinics, and donations to ensure data integrity, automate capacity checks, and streamline health tracking.

1.3. Project Objective and Justification

Animal welfare is a societal responsibility. This project aims to digitize the entire process of rescuing and adopting stray animals. The developed system provides a logistical tracking mechanism; it optimizes the capacity management of temporary shelters, systematizes vaccinations and veterinary visits, and ensures that donations reach the right targets. This tracking system minimizes operational errors and plays a life-saving role.

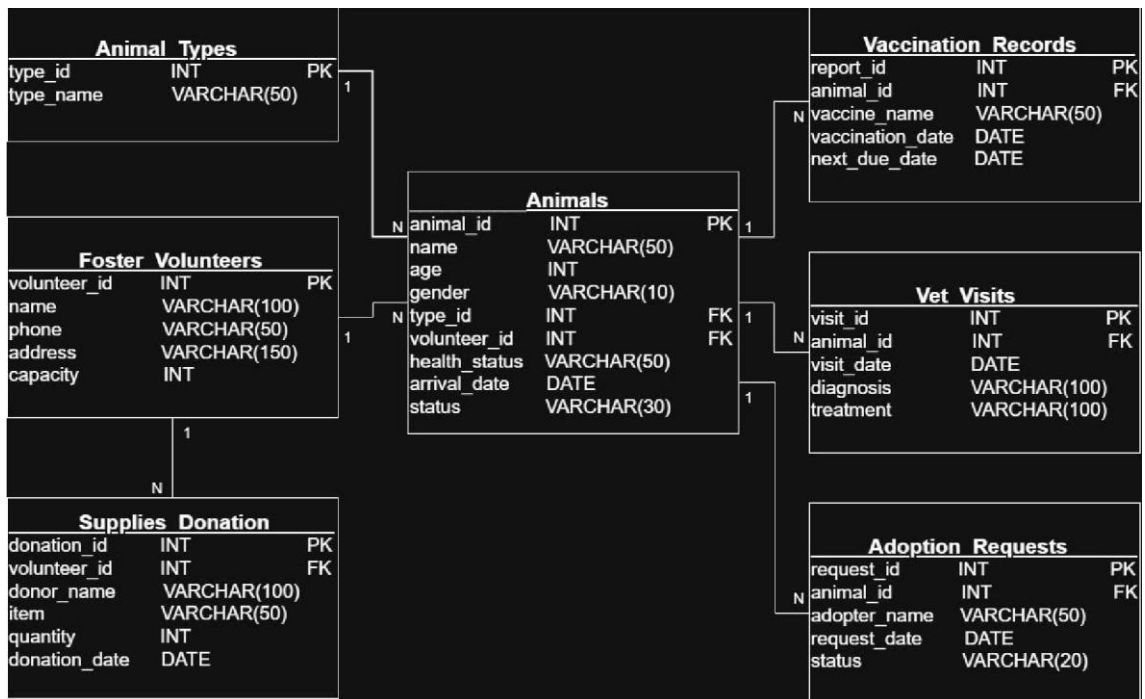
2. DATABASE DESIGN AND ARCHITECTURE

The system is built on a relational database model. Relationships between core entities are normalized up to Third Normal Form (3NF) to reduce redundancy and ensure data integrity. The schema is designed with 7 tables to accurately represent real-world operations and support efficient querying and reporting.

Table 2.1. Table Schemas

Table Name	Description	Key Relationships
Animal_Types	Master data for animal species/breeds.	1:N with Animals
Foster_Volunteers	Volunteer contact info and max safe capacity limit.	1:N with Animals & Supplies_Donations
Animals	Core identity, age, status, and current assignment.	FK to Foster_Volunteers & Animal_Types
Vaccination_Records	Logs of applied vaccines and upcoming due dates.	FK to Animals
Vet_Visits	Clinical diagnosis and treatment histories.	FK to Animals
Adoption_Requests	Tracking potential adopters and request statuses.	FK to Animals
Supplies_Donations	Tracking donated items assigned to specific volunteers.	FK to Foster_Volunteers

Figure 2.1. Entity Relationship Diagram (ERD)



3. TECHNICAL IMPLEMENTATION FEATURES

3.1. Business Rules Implemented in SQL

- **Capacity Integrity:** A volunteer's capacity must always be > 0 (CHECK constraint).
- **Age Validation:** An animal's age cannot be negative (CHECK ≥ 0).
- **Status Control:** Animal statuses are restricted to specific domains (e.g., 'Fostered', 'Adopted', 'Under Treatment').
- **Referential Integrity:** volunteer_id consistency ensures no animal is assigned to a non-existent volunteer.

3.2. SQL Features Used in the System

To analyze and manage this data effectively, we will utilize the following SQL techniques:

- **INNER / LEFT JOIN:** To link animals with their specific volunteers, types, and medical records.
- **GROUP BY + HAVING:** To monitor volunteer capacity utilization and donation distributions.
- **WITH (CTE):** To build structured analytical queries for upcoming vaccination deadlines.
- **VIEWS:** To create easily accessible virtual tables for daily operations (e.g., "Ready for Adoption" view).
- **ORDER BY & Aggregation:** Using COUNT(), MAX(), and date filtering for monthly reporting.

3.3. Sample Scenarios and SQL Queries

- **Scenario 1: The Active Roster (INNER JOIN)**

Lists all currently fostered animals along with their type and responsible volunteer's contact information.

```
SELECT
  a.name AS Animal_Name,
  t.type_name AS Species,
  a.age,
  f.name AS Foster_Name,
  f.phone
FROM Animals a
INNER JOIN Foster_Volunteers f
  ON a.volunteer_id = f.volunteer_id
INNER JOIN Animal_Types t
  ON a.type_id = t.type_id
WHERE a.status = 'Fostered'
ORDER BY t.type_name, a.name;
```

- **Scenario 2: Capacity Control (GROUP BY + HAVING)**

Identifies volunteers who are currently at or over their maximum safe fostering capacity.

```
SELECT
  f.name AS Volunteer_Name,
  f.capacity AS Max_Capacity,
  COUNT(a.animal_id) AS Current_Fosters
FROM Foster_Volunteers f
LEFT JOIN Animals a
  ON f.volunteer_id = a.volunteer_id
GROUP BY f.name, f.capacity
HAVING COUNT(a.animal_id) >= f.capacity;
```

- **Scenario 3: Urgent Vaccination Pipeline (CTE)**

Uses a Common Table Expression (CTE) to isolate upcoming vaccination needs due within the next 30 days.

```
WITH Upcoming_Vaccines AS (  
  SELECT  
    animal_id,  
    vaccine_name,  
    next_due_date  
  FROM Vaccination_Records  
  WHERE next_due_date BETWEEN  
CURRENT_DATE  
  AND CURRENT_DATE +  
INTERVAL '30 days'  
)  
  
SELECT  
  a.name AS Animal_Name,  
  f.name AS Volunteer_Name,  
  u.vaccine_name,  
  u.next_due_date  
FROM Animals a  
INNER JOIN Upcoming_Vaccines u  
  ON a.animal_id = u.animal_id  
INNER JOIN Foster_Volunteers f  
  ON a.volunteer_id = f.volunteer_id  
ORDER BY u.next_due_date ASC;
```

4. WEB APPLICATION & CLOUD DEPLOYMENT

4.1 Technology Stack

To transform the database project into a fully interactive and accessible system, modern cloud and web technologies were integrated with the SQL infrastructure.

The backend database was implemented using PostgreSQL and hosted on Supabase, providing a scalable and reliable cloud-based relational database environment.

The application interface was developed using Streamlit, allowing users to interact with the system through a web browser without directly writing SQL queries.

Python was used as the middleware layer for handling database connections, query execution, and real-time data operations between the user interface and the PostgreSQL database.

<u>Technology</u>	<u>Purpose</u>
PostgreSQL	Relational database management
Supabase	Cloud database hosting
Streamlit	Interactive web application interface
Python	Backend logic and database connectivity

4.2 Cloud Deployment

Unlike traditional local SQL projects, the developed system was deployed as a live cloud-based application.

The PostgreSQL database is hosted remotely through Supabase, while the Streamlit application is publicly accessible through a deployed web interface.

This architecture enables real-time interaction with the database, allowing users to perform operations such as registering animals, assigning foster volunteers, and managing vaccination records directly through the browser.

The deployed system demonstrates a practical implementation of database technologies in a real-world environment.

4.3 Real-Time Database Operations

The web application performs live database operations through the Streamlit interface connected to the PostgreSQL database.

The system supports real-time DML (Data Manipulation Language) operations including INSERT, UPDATE, and SELECT queries.

Examples of implemented operations include:

- Registering new foster volunteers (INSERT)
- Registering rescued animals (INSERT)
- Adding vaccination records (INSERT)
- Assigning animals to volunteers (UPDATE)
- Generating alerts and reports (SELECT)

These operations are executed dynamically through the user interface without requiring manual SQL execution by the end user.

5. SYSTEM INTERFACE & OUTPUTS

Figure 5.1 — System Overview

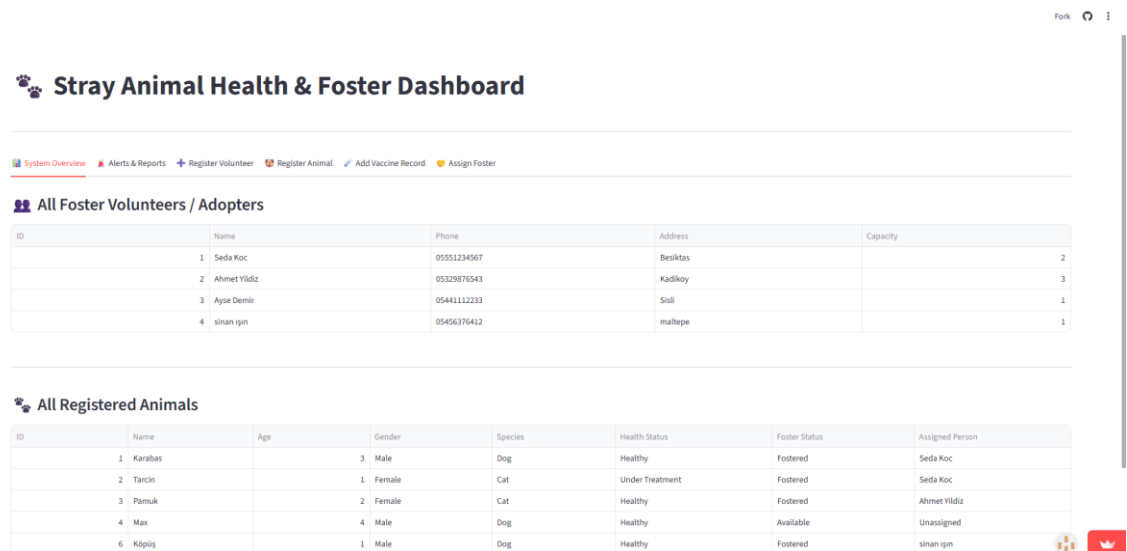


Figure 5.1 demonstrates the main dashboard of the system, including foster volunteer management, registered animal monitoring, and real-time operational overview.

Figure 5.2 — Alerts & Reports

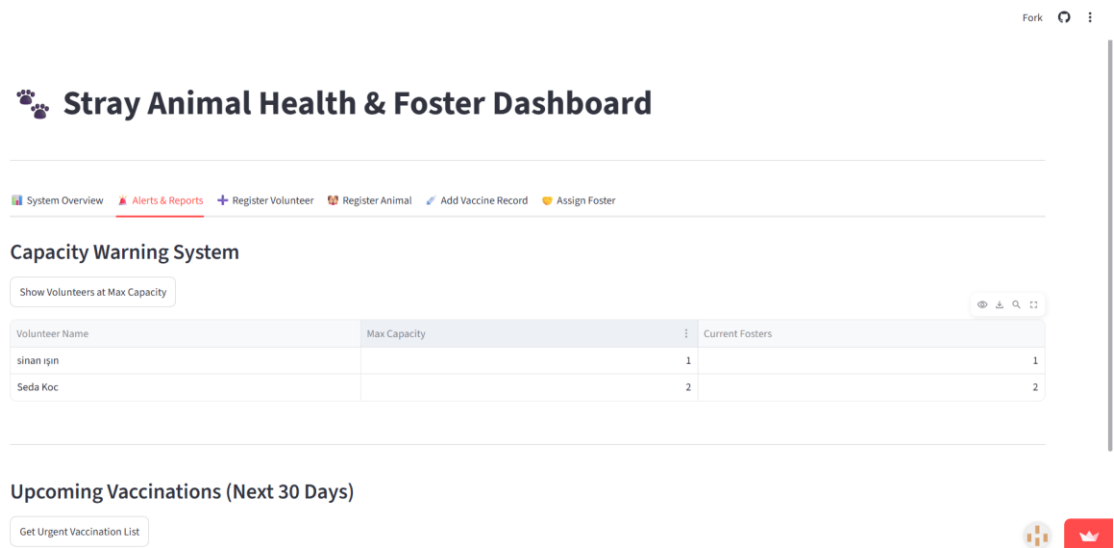


Figure 5.2 demonstrates the alert and reporting module used for identifying volunteers operating at maximum foster capacity and monitoring upcoming vaccination schedules.

Figure 5.3 — Register Volunteer

Stray Animal Health & Foster Dashboard

System Overview Alerts & Reports Register Volunteer Register Animal Add Vaccine Record Assign Foster

Add New Foster Volunteer / Adopter

Full Name

Phone Number

Address

Max Capacity

1

Save to Database

Figure 5.3 demonstrates the volunteer registration interface, which allows users to insert new foster volunteer records into the PostgreSQL database through the Streamlit web application.

Figure 5.4 — Register Animal

The screenshot shows the 'Stray Animal Health & Foster Dashboard' with a navigation bar containing links for System Overview, Alerts & Reports, Register Volunteer, Register Animal (highlighted), Add Vaccine Record, and Assign Foster. The main section is titled 'Register a New Rescued Animal' and contains a form with the following fields: 'Animal Name' (text input), 'Age' (text input with '0' entered), 'Gender' (dropdown menu with 'Male' selected), 'Species' (dropdown menu with 'Dog' selected), and 'Health Status' (dropdown menu with 'Healthy' selected). A 'Save to Database' button is located at the bottom of the form. The interface includes a 'Fork' button in the top right corner and a GitHub logo in the bottom right corner.

Figure 5.4 demonstrates the rescued animal registration interface, allowing users to insert animal information, species type, and health status records into the PostgreSQL database.

Figure 5.5 — Add Vaccine Record

The screenshot shows the 'Stray Animal Health & Foster Dashboard' with a navigation bar containing links for System Overview, Alerts & Reports, Register Volunteer, Register Animal, Add Vaccine Record (highlighted), and Assign Foster. The main section is titled 'Add New Vaccination Record' and contains a form with the following fields: 'Select Animal' (dropdown menu with 'Karabas (ID: 1)' selected), 'Vaccine Name (e.g. Rabies, Feline Viral)' (text input), 'Vaccination Date' (text input with '2026/06/04' entered), and 'Next Due Date' (text input with '2027/06/04' entered). A 'Save Vaccine Record' button is located at the bottom of the form. The interface includes a 'Fork' button in the top right corner and a GitHub logo in the bottom right corner.

Figure 5.5 demonstrates the vaccination management interface, where users can assign vaccine records and upcoming due dates to registered animals in the PostgreSQL database.

Figure 5.6 — Assign Foster

The screenshot displays the 'Stray Animal Health & Foster Dashboard' with a navigation bar containing links for System Overview, Alerts & Reports, Register Volunteer, Register Animal, Add Vaccine Record, and Assign Foster (highlighted). The main content area is divided into two sections. The first section, 'Assign Animal to a Person', includes three dropdown menus: 'Select Unassigned Animal' (showing 'Max (ID: 4)'), 'Select Person' (showing 'Ahmet Yildiz (ID: 2)'), and 'Assignment Type' (showing 'Fostered (Temporary)'). A 'Confirm Assignment' button is located below these menus. The second section, 'Unassign Animal (Return to Shelter)', features a 'Select Animal to Unassign' dropdown menu showing 'Karabas (ID: 1)' and an 'Unassign' button. The interface is clean and modern, with a light gray background and white form elements.

Figure 5.6 demonstrates the animal assignment interface, where unassigned animals can be matched with foster volunteers through real-time UPDATE operations in the PostgreSQL database.

6. CONCLUSION

The designed Stray Animal Foster & Health Tracking System aims to prevent confusion in the field by offering a database-driven solution. Thanks to the system's modular structure, future integration with mobile applications or automated notification systems (vaccination reminders via SMS/email) is possible. This infrastructure forms a strong foundation for sustainable animal welfare management.